Jobblo – Security Review Report

Full review of every file and folder (backend, web app, mobile app, server and deploy config), done after the first round of fixes · 26 Oct 2026 · committed locally on branch fixed/secuity, not pushed

66	5	64	0	1
findings	high / critical	fixed + tested	being fixed	owner action

How it was done: eight independent reviewers each took one area (money, login, access to data, input and uploads, chat and sockets, web app, mobile app, server and deploy). They re-checked every earlier fix by trying to break it, and reported only problems they could show with a concrete path through the code. Every fix in this report has a test that fails on the old code and passes on the new.

Result: no critical code problem was found. The earlier fixes hold; the few partial ones are listed here and fixed. The only critical item left is the Azure secret in git history, which the owner must rotate.

Tests at the time of writing: backend 1,985 passed (one flaky queue test fixed since); web E2E 103 passed + 3 former failing tests now passing; mobile 13 checks and typecheck passing. The web typecheck is now clean (102 old errors fixed) and blocks CI; lint errors went from 209 to 56.

Money: SafePay, payouts, disputes, chargebacks

ID	Severity	Problem, where, fix	Status
F1	High	**A re-resolved dispute could refund the customer twice** `controllers/admin/disputesAdminController.js` **What goes wrong:** A refund times out but happened; the admin re-resolves with other amounts and a second refund goes through (133 paid out of 103 collected). **Fix:** Refuse a new refund while another refund for the same dispute is pending, succeeded or unknown.	Fixed
F2	High	**A won chargeback blocked the payout forever** `services/payout/releasePayoutToProvider.js, disputesAdminController.js, providerConnectTimeout.js` **What goes wrong:** Stripe keeps charge.disputed = true after the platform wins; every payout attempt failed and the provider was never paid. **Fix:** Block only open or lost Stripe disputes (dispute status read from Stripe).	Fixed
F3	Medium	**Dispute on a payout with unknown outcome froze the money** `safePayStateService.js` **What goes wrong:** The retry job and the admin both refused, so neither could finish. **Fix:** A dispute cannot be opened or reopened while that payout is unresolved (max. 24 h).	Fixed
F4	Medium	**A message or status change could undo a resolved dispute** `disputeController.js, disputesAdminController.js` **What goes wrong:** Read-then-save overwrote "resolved" with an active status; the money stayed frozen. **Fix:** Conditional update on the status that was read; 409 if it changed.	Fixed

F5	Medium	**Chargeback incidents could never close** `stripePaymentIncidentController.js` **What goes wrong:** Reconciled only at the full charge (103) although the transfer was 97; a lost chargeback before payout had no way out. **Fix:** Reconcile at min(charge, transfer); new admin action "close lost chargeback" (API only, no button yet).	Fixed
F6	Low	**Two admins could take back more than the chargeback** `stripePaymentIncidentController.js` **What goes wrong:** The cap was read before the reversal; two approvals at once both passed. **Fix:** Short lock on the Stripe dispute during the reversal.	Fixed
F7	Low	**A transfer found later was only half recorded** `releasePayoutToProvider.js` **What goes wrong:** Payout marked paid but Payment and money history not updated. **Fix:** One shared "record recovered transfer" step for both paths.	Fixed
AUD	Medium	**Admin audit entries were silently lost** `models/AdminActivity.js` **What goes wrong:** 19 action names (dispute resolve, refunds, chargeback recovery, shop changes) were not allowed, so the audit record was dropped without error. **Fix:** Names added; a test fails if code logs an action the model does not allow.	Fixed

Login, accounts and 2FA

ID	Severity	Problem, where, fix	Status
A1	Medium	**Revoked sessions kept receiving push notifications** `models/Session.js` **What goes wrong:** After a password change, revoke or admin deactivation the stolen phone still got chat notifications. **Fix:** Ending any session switches off its push tokens.	Fixed
A2	Medium	**An attacker could lock the real owner out of login** `controllers/auth/loginController.js` **What goes wrong:** 100 failures for one email from many IPs blocked everyone, including the owner. **Fix:** The email-wide limit only applies to clients that failed themselves.	Fixed
A3	Medium	**Unlimited password guesses with a stolen session** `controllers/users/profileController.js` **What goes wrong:** Email change and account delete checked the password with no limit; a hit gave full takeover. **Fix:** One shared check with a limit of 10 wrong tries per 15 minutes.	Fixed
A4	Medium	**Anonymous logins could stall the server** `loginController.js, profileController.js, admin user creation` **What goes wrong:** Raw bcrypt calls skipped the bounded password queue. **Fix:** All password checks and hashes use the bounded queue (503 when busy).	Fixed

A5	Low	**Pre-registered account: attacker payout/BankID survived the owner's reset** `controllers/auth/passwordController.js` **What goes wrong:** An attacker who registered your email could keep their bank account and BankID on it. **Fix:** New accounts without email proof are cleared on reset; accounts created before 26 Oct 2026 are not touched. Real fix: email verification at sign-up.	Fixed
A6	Low	**Anyone could fill the reset-code budget** `controllers/auth/authHelpers.js` **What goes wrong:** 20 wrong guesses blocked the owner's password reset. **Fix:** Only misses count, per account and IP.	Fixed
A7	Low	**A 2FA or backup code could be used twice in parallel** `services/auth/mfa.js` **What goes wrong:** Parallel requests all accepted the same code. **Fix:** Atomic use of the code; only one request wins.	Fixed
A8/Z4	Medium	**A superAdmin could change another user's email** `profileController.js` **What goes wrong:** Changing another admin's email, then resetting the password, took over the account with no audit. **Fix:** Only the owner can change their own email; admin changes of verified/trusted are audit-logged.	Fixed
A9	Low	**Endless 2FA guessing without warning** `controllers/mfaController.js` **What goes wrong:** 6 tries per 15 min forever, nobody told. **Fix:** After 20 wrong codes in 24 h the session ends and the owner gets an email.	Fixed

Access to other users' data

ID	Severity	Problem, where, fix	Status
Z1	Medium	**List search showed non-public listings** `controllers/users/searchController.js` **What goes wrong:** Search-all returned drafts and finished jobs inside public lists. **Fix:** Only public (or own) listings are filled in.	Fixed
Z2	Medium-High	**Adding people to a list revealed their email** `controllers/listController.js` **What goes wrong:** Anyone could add strangers as contributors and then read their email addresses. **Fix:** Only the list owner sees emails; max 20 existing users per call.	Fixed
Z3	Medium	**Either party could permanently delete a shared chat** `chatController.js, messageController.js` **What goes wrong:** Evidence of abuse could be wiped before a report. **Fix:** Deleting now only hides it for you.	Fixed
Z5/I5	Low	**Private evidence links could be re-published** `utils/azureUpload.js` **What goes wrong:** A stripped evidence link pasted as a review photo got a fresh signed link for every viewer. **Fix:** Private links refused in review photos, portfolio and listing images.	Fixed

I6	Low	**Image links could point anywhere** `manageController.js, reviewPhotos.js, profileController.js` **What goes wrong:** External tracking images; the 6-image limit could be bypassed. **Fix:** Only images from Jobblo's own storage, max 6 per listing.	Fixed

Chat, sockets and real-time

ID	Severity	Problem, where, fix	Status
S1	Medium	**Socket event flood** `sockets/chat.socket.js` **What goes wrong:** Repeated "mark as read" made the victim's app refetch until it was rate-limited. **Fix:** 20 events per 10 s per socket; broadcast only when something changed.	Fixed
S2	Medium	**Logged-out sockets survived when Redis was down** `sockets/rooms.js` **What goes wrong:** Revocation only travelled through Redis. **Fix:** Local sockets are closed first, then the other servers.	Fixed
S3	Medium	**Sockets lived forever** `chat.socket.js, frontend socket.ts` **What goes wrong:** Expired tokens and expired sessions kept their socket. **Fix:** Socket closes when the token expires; the web client reconnects and re-joins the chat.	Fixed
S4	Medium	**Deactivated accounts could still use sockets** `chat.socket.js, middleware/auth.js` **What goes wrong:** The socket handshake did not check account status. **Fix:** Same account check as the HTTP API.	Fixed
S5	Medium	**Unlimited sockets per user** `chat.socket.js` **What goes wrong:** Each connect scanned all chats and notified every partner. **Fix:** Max 10 sockets per user; presence work only on the first.	Fixed
S6	Medium	**Block list missing on contract steps** `chatController.js, orderController.js, safepayController.js` **What goes wrong:** A blocked user could still accept, set a price or award a contract. **Fix:** Block check on every step; price changes rate-limited.	Fixed
S7	Low	**Blocked users could see when you were online** `chat.socket.js` **What goes wrong:** Presence went to blocked partners. **Fix:** Filtered both ways.	Fixed
S8	Low	**Raw objects in socket chat ids** `chat.socket.js` **What goes wrong:** Objects reached database queries. **Fix:** Only valid id strings accepted.	Fixed
S9	Low	**1 MB socket messages** `bootstrap/socket.js` **What goes wrong:** Large frames for memory pressure. **Fix:** Max 64 KB.	Fixed

S10	Low	**pm2 cluster without Redis** `server.js, cluster.js` **What goes wrong:** Logout did not reach other workers. **Fix:** Extra workers refuse to start without REDIS_URL.	Fixed
S11	Low	**Unused message endpoint with no checks** `routes/messages.js` **What goes wrong:** No length or type checks. **Fix:** Endpoint removed (no client used it).	Fixed
S12	Low	**First application message not checked** `orderController.js` **What goes wrong:** Up to 256 KB of text. **Fix:** Text only, max 5000 characters.	Fixed
S13	Info	**No leave-chat handler** `chat.socket.js` **What goes wrong:** Sockets stayed in every chat room. **Fix:** leave-chat handler added; dead web emit removed.	Fixed

Input handling and uploads

ID	Severity	Problem, where, fix	Status
I1	High	**Log masking could freeze the server (ReDoS)** `utils/sanitizer.js` **What goes wrong:** One 120 KB value in an error message blocked the server for 84 seconds. **Fix:** Anchored pattern + 20 000-character cap: now ~1 ms; also masks 412-34-567 and kari%40... forms.	Fixed
I2	High	**Upload forms had no field limits** `multer setups in middleware/ and routes/` **What goes wrong:** A 40-byte field "tags[50000000]" used 1.2 GB of memory and 5.6 s per request. **Fix:** Field count/size limits and an array-length guard on every upload route.	Fixed
I3	Medium	**Anonymous users could fill Redis via the cache** `utils/cache.js` **What goes wrong:** Random query strings stored new copies of large responses. **Fix:** Cache key on the path for routes that ignore the query.	Fixed
I4	Medium	**Huge images cost a lot of CPU** `middleware/upload.js` **What goes wrong:** A 16000x16000 PNG took 1.3 s to process. **Fix:** Pixel limit on image decoding.	Fixed
I7	Low	**Two lists had no page cap** `browseController.js, searchController.js` **What goes wrong:** A huge page number forced a full scan. **Fix:** Shared page limit (max 1000).	Fixed
I8	Low	**Error log stored search terms from the URL** `utils/serverError.js` **What goes wrong:** Emails or addresses in the query string were logged. **Fix:** Log the path only.	Fixed

I9	Low	**AI image route trusted the file type from the browser** `routes/ai.js` **What goes wrong:** Any file up to 5 MB was sent to the paid AI service. **Fix:** Real file type checked from the bytes.	Fixed
I10	Info	**Anyone can submit fake client error reports** `routes/errors.js` **What goes wrong:** Fake entries in the developer console. **Fix:** Client codes are always CLIENT_/FRONTEND_-prefixed and stored as warnings, so they cannot pose as server errors.	Fixed

Web app

ID	Severity	Problem, where, fix	Status
W1	Low-Medium	**Private chat text in monitoring** `frontend/src/lib/rum.ts` **What goes wrong:** Clicked chat rows (names, message start) reached the developer console with error trails. **Fix:** Only labels, never visible text.	Fixed
W2	Low	**Old success page acted on load** `ContactSuccessPage.tsx` **What goes wrong:** Unused page created a job request from URL parameters. **Fix:** Page and route removed.	Fixed
W3/X8	Low	**No script content-security policy** `deploy/nginx/jobblo-security-headers.conf` **What goes wrong:** Any XSS could use the session. **Fix:** Report-only CSP + Permissions-Policy added; switch to enforcing after watching.	Fixed
W4	Low	**Profile website not validated** `profileController.js` **What goes wrong:** Any text as a link. **Fix:** Must be an http(s) address.	Fixed
E2E	Low	**6 bugs found by the new E2E tests** `CompletedJobPage, OrderRequestModal, ProviderOrderDetailPage, Header, DisputeDetailPage, vite.config.ts` **What goes wrong:** Leaving the site from a deep link, inaccessible dialogs and stars, noisy log, unlinked labels, dev reloads. **Fix:** All fixed; 3 former failing tests now pass.	Fixed

Mobile app

ID	Severity	Problem, where, fix	Status
P2	Medium	**Login tokens written to the nginx access log** `deploy/nginx/*.conf` **What goes wrong:** Every app sign-in logged a valid token (and OAuth codes). **Fix:** Log format without query strings. Owner: purge old logs.	Fixed

P3	Medium-Low	**Token refresh could switch to another account** `jobblo-app/src/api/client.ts` **What goes wrong:** After a failed logout, the next user could silently become the previous one. **Fix:** Refreshed token must belong to the signed-in user.	Fixed
P4	Low	**Notifications stayed in the tray after logout** `jobblo-app/src/store/authStore.ts` **What goes wrong:** The next person on the phone could read them. **Fix:** Tray cleared on logout.	Fixed
P5	Low	**Unneeded Android permissions** `jobblo-app/app.json` **What goes wrong:** Overlay, storage and microphone permissions in builds. **Fix:** Blocked in app config. Owner: delete the old android/ folder.	Fixed
P6	Medium	**Login token still travels in the jobblo:// link** `oauthReturn.js, oauthAuthSession.ts` **What goes wrong:** Another Android app claiming the jobblo scheme could catch it. **Fix:** The link now carries a 60-second one-time code; the app exchanges it with its PKCE verifier (S256) for the session, so an intercepted link is useless. Deploy the backend before the new app build.	Fixed
P7	Info	**npm audit findings in the app** `jobblo-app` **What goes wrong:** Build tools only (Expo CLI, Tailwind), nothing shipped. **Fix:** Wait for the next Expo update; do not run audit fix --force.	Report only

Server, deploy and CI

ID	Severity	Problem, where, fix	Status
X1	Medium	**Deploy did not roll back on a restart failure** `deploy.sh` **What goes wrong:** A failing npm ci / pm2 left the server half-updated. **Fix:** set -E so the rollback always runs.	Fixed
X2	Medium	**Detached HEAD deployed main without CI** `deploy.sh` **What goes wrong:** A manual checkout skipped the CI gate. **Fix:** Refuses to deploy without a branch.	Fixed
X3	Medium	**Feature branches deployed without checks** `deploy.sh, ci.yml` **What goes wrong:** Only main needed green CI. **Fix:** CI on every push; every deploy needs green CI.	Fixed
X4	Medium	**Unlimited anonymous support tickets** `routes/support.js` **What goes wrong:** ~96 000 tickets per day per IP. **Fix:** 10 per hour.	Fixed
X5	Low	**pm2 killed the API mid-shutdown** `ecosystem.config.js` **What goes wrong:** 12 s timeout shorter than the 15 s cluster drain. **Fix:** 16 s.	Fixed

X6	Low	**Scheduler kept running during shutdown** `bootstrap/background.js` **What goes wrong:** Money jobs could start and lose the database mid-run. **Fix:** Scheduler stops first.	Fixed
X7	Low	**Session secret could equal the JWT secret** `config/session.js` **What goes wrong:** Same key for two purposes. **Fix:** Refused in production.	Fixed
X9	Low	**Audit only for the backend** `ci.yml` **What goes wrong:** Frontend/app dependency problems unnoticed. **Fix:** Audit steps for frontend and app.	Fixed
X10	Low	**CI permissions too narrow for PR checks** `ci.yml` **What goes wrong:** Path filter and secret scan fail on PRs. **Fix:** pull-requests: read. Owner: GITLEAKS_LICENSE secret.	Fixed
X12	Low	**Login limits switched off when Redis failed** `middleware/rateLimiter.js` **What goes wrong:** No brute-force limit during a Redis outage. **Fix:** Login, OTP and 2FA limiters fail closed.	Fixed
X11/C1	Critical (owner)	**Azure secret in git history** `infra/provider.tf (commits 24bd7b24..4aa1e20a)` **What goes wrong:** The secret was in the published code until 24 Oct 2026. **Fix:** Rotate in Entra ID, review sign-ins, then allowlist in gitleaks.	Owner action
X13	Low	**Docker image not production-grade** `backend/Dockerfile` **What goes wrong:** Only matters if Docker is ever used in production. **Fix:** Image is production by default (npm ci --omit=dev, NODE_ENV=production); compose builds a dev variant; TRUST_PROXY documented.	Fixed

What the owner must do

1. Rotate the Azure service-principal secret (C1) and the leaked .env keys (C4); review Azure sign-ins.
2. Server: HOST=127.0.0.1; SESSION_SECRET set and different from JWT_SECRET; both Stripe webhook secrets and both secret keys.
3. Server: start the API with backend/ecosystem.config.js (pm2 fork mode, WEB_CONCURRENCY=auto, REDIS_URL set); once: pm2 delete jobblo-api && pm2 start ecosystem.config.js && pm2 save.
4. nginx: install the security-headers snippet and the new log format, nginx -t, reload; purge old access/error logs (they contain login tokens).
5. Azure Storage: create a private container and set AZURE_PRIVATE_CONTAINER; connection string with AccountName + AccountKey.
6. GitHub: set the GITLEAKS_LICENSE secret; install and log in gh on the server (CI gate in deploy.sh).
7. Mobile: new EAS build (secure storage, permissions); EXPO_PUBLIC_API_URL https for preview; decide Sign in with Apple; delete the old android/ folder.
8. Google Cloud: restrict the browser Maps key (websites + APIs) and set quota alerts.
9. Deploy the backend before releasing the next app build (the app now refuses token links and expects the one-time code); remove the old token fallback once old app builds are gone.

Details of every fix are in Jobblo-Security-Fix-Log.pdf. Generated from docs/security/review-report.html.